

JIMS-AI Complete Specification v10

Generated from the current JIMS-AI codebase and production architecture.

Status date: June 1, 2026

This specification reflects the current JIMS-AI codebase, deployed services, database schema, and implementation plan. It consolidates the v8 cognitive architecture, v9 product expansion, `solution_v1.md`, the prediction-trap design notes, and the production deployment state now represented in the repository.

1. Product Definition

JIMS-AI is a memory-centric AI runtime. It is not designed to compete with frontier foundation models on raw general intelligence. It is designed to own durable project memory, provenance, source-backed answers, graph reasoning context, training workflow, and human-approved improvement.

The primary user experience is a persistent multi-thread chat interface backed by structured memory. The user can ask questions, receive live responses, submit feedback, mark answers as learnable, and remove learned memory. The system records training signals and exposes them through a training console.

2. Core Architectural Decision

Sentence-transformer inference must not run inside AWS Lambda. Lambda remains lightweight and reliable. Semantic embedding runs in a separate always-reachable service, currently deployed as a Hugging Face Space.

The runtime rule is:

1. Use the live sentence-transformer embedding service for semantic vectors.
2. Use deterministic exact, structured, and graph retrieval alongside semantic retrieval.
3. Use hash embedding only as a rare recoverable fallback.
4. Mark fallback vectors as recoverable so the training service can later re-embed them through `/v1/autonomous/reembed-hash`.

3. Deployed Production Topology

Live frontend:

`https://jimsai.vercel.app`

Live AWS Lambda API:

`https://7x27vovhmfnhcymm5ox3qiw4fy0agvcy.lambda-url.us-east-1.on.aws`

Live Render training service:

<https://jimsai-training-service.onrender.com>

Live Hugging Face embedding service:

<https://jimstechai-jimsai-embedding-service.hf.space>

Production flow:

Vercel Next.js UI

- > AWS Lambda FastAPI API
- > Supabase Auth and Postgres
- > Hugging Face sentence-transformer embedding service
- > Cloudflare Vectorize
- > Neo4j Aura graph memory
- > Redis session/cache state
- > Groq only when deterministic runtime needs bounded model help

External cron

- > Render training service
- > Supabase state
- > Hugging Face embedding service
- > Kaggle GPU artifact packaging

4. Frontend

The frontend is a Next.js application deployed to Vercel with GitHub auto-deploy.

Public frontend variables only:

NEXT_PUBLIC_API_BASE_URL
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY

The production chat UI supports:

1. User sign-in through Lambda/Supabase.
2. Persistent chat threads.
3. Thread selection.
4. New-thread creation.
5. Thread deletion.
6. Loading server-persisted messages.
7. Sending thread_id with each query.
8. Feedback submission.
9. Learn-this feedback.
10. Unlearn/delete memory.

Thread history is persisted in Supabase through Lambda. Local browser state is only a responsiveness layer and is not the system of record.

5. AWS Lambda API

Lambda hosts the production FastAPI gateway from `services/api-gateway`.

Required production settings:

```
timeout=120 seconds
memory=1024 MB
```

Lambda responsibilities:

1. Supabase auth proxy and bearer-token validation.
2. `/v1/query`.
3. `/v1/query/stream`.
4. `/v1/training/ingest`.
5. `/v1/feedback`.
6. `/v1/chat/threads`.
7. `/v1/chat/threads/{thread_id}/messages`.
8. `/v1/chat/threads/{thread_id} delete`.
9. `/v1/memory/insert`.
10. `/v1/memory/update`.
11. `/v1/memory/delete`.
12. Training dashboard and panel reads.
13. Review actions.
14. Provider readiness.
15. Canvas, invention, and Kaggle scheduling endpoints.
16. Deterministic Python sandbox execution.
17. Solver-backed math validation.
18. Memory statistics.
19. Audit event inspection.

Lambda does not install PyTorch or sentence-transformers. It calls the embedding service when semantic search is needed and degrades to exact/structured/graph retrieval if the embedding service is temporarily unavailable.

6. Query Runtime

Final query flow:

```
Frontend
-> Lambda /v1/query
  -> validate Supabase session
  -> compile query into semantic IR
  -> resolve intent and capability route
  -> use exact and structured memory lookup
  -> call embedding service for sentence-transformer vector
  -> query Vectorize for top-k semantic matches
  -> hydrate records from Supabase
  -> expand graph context from Neo4j
  -> score exact, semantic, graph, trust, and lexical signals
  -> render deterministic answer where possible
  -> call Groq only when needed
  -> persist chat messages, retrieval events, traces, and feedback hooks
  -> return answer to frontend
```

The runtime supports multi-intent routing through the semantic compiler, capability planner, memory retrieval, graph context, canvas/invention activation hints, and bounded model bridges.

7. Embedding Service

The production embedding service runs on Hugging Face Spaces using CPU Basic hardware. It currently uses:

```
intfloat/multilingual-e5-small
dimension=768
```

Required endpoints:

```
GET /health
GET /ready
POST /v1/embed
POST /v1/embed-batch
POST /v1/encode
GET /v1/artifact/current
POST /v1/reload-artifact
```

Protected endpoints require bearer token auth. Hash fallback exists only for rare recovery cases and should set metadata that allows the training service to re-embed later.

The live verification on June 1, 2026 returned a real sentence-transformer vector:

```
model=intfloat/multilingual-e5-small
dimension=768
fallback=false
```

8. Render Training Service

Render runs one bounded web service:

```
jimsai-training-service
```

It auto-deploys from GitHub through `render.yaml`.

Primary endpoints:

```
GET /health
POST /v1/autonomous/discover
POST /v1/autonomous/ingest-batch
POST /v1/autonomous/evaluate
POST /v1/autonomous/plan
POST /v1/autonomous/report
POST /v1/autonomous/kaggle/package
POST /v1/autonomous/reembed-hash
```

All autonomous endpoints require:

```
Authorization: Bearer <JIMS_RENDER_AGENT_TOKEN>
```

The training service must be stateless between requests. It stores run state, jobs, reports, batches, panel items, signatures, and artifacts in Supabase.

Current required Render secrets:

```
JIMS_RENDER_AGENT_TOKEN
JIMS_EMBEDDING_SERVICE_URL
JIMS_EMBEDDING_SERVICE_TOKEN
SUPABASE_URL
SUPABASE_SERVICE_KEY
SUPABASE_ANON_KEY
CF_ACCOUNT_ID
CF_VECTORIZE_INDEX
CF_VECTORIZE_API_TOKEN
NEO4J_URI
NEO4J_USER
NEO4J_PASSWORD
KAGGLE_USERNAME
KAGGLE_API_TOKEN
KAGGLE_DATASET_OWNER
GRQ_API_KEY
```

Render does not read the local `.env` file. Any `sync: false` value in `render.yaml` must be set in the Render dashboard or through the Render API.

9. Supabase Schema

`infrastructure/postgres/supabase.sql` is the single source of truth for the production schema.

Core runtime tables:

```
signatures
execution_traces
training_panel_items
workspaces
workspace_members
jimsai_events
request_audit
sppe_pairs
```

```
memory_signatures
memory_chunks
retrieval_events
retrieval_misses
user_feedback
chat_threads
chat_messages
```

Autonomous and artifact tables:

```
autonomous_runs
autonomous_jobs
ingestion_cursors
training_batches
training_artifacts
evaluation_reports
approval_queue
```

Operational tables:

```
workspace_metrics
workspace_quotas
query_patterns
user_preferences
workspace_adapters
provider_state
system_metrics
```

The schema is idempotent and safe to re-run in Supabase SQL Editor. The active production database has already been updated with the current schema.

10. Hybrid Retrieval

JIMS-AI uses four retrieval signals:

```
semantic_score * 0.55
lexical_score  * 0.20
graph_score    * 0.15
trust_score    * 0.10
```

The retrieval stack is:

1. Exact and structured Supabase lookup.
2. Sentence-transformer vector search through Cloudflare Vectorize.
3. Graph expansion through Neo4j.
4. Trust/provenance weighting from human review, source trust, and previous successful usage.

Exact facts must still win when appropriate. Semantic retrieval is used for paraphrase and multilingual memory. Graph expansion is used for causal and dependency context.

11. Memory and Training Lifecycle

Healthy lifecycle:

1. User chats.
2. Lambda records query, answer, sources, confidence, trace, and thread.
3. User submits feedback or marks an answer as learnable.
4. Training ingestion stores signatures and SPPE pairs.
5. Autonomous service discovers, ingests, evaluates, reports, and packages bounded batches.
6. Kaggle trains candidate artifacts offline.
7. Evaluation reports compare old and new artifacts.
8. Human approves or rejects activation.
9. Embedding service reloads the approved active artifact.
10. Future retrieval improves.

Activation is never automatic. Humans approve artifacts before production activation.

12. Kaggle Role

Kaggle is the offline GPU lab. It is used for:

1. Encoder fine-tuning.
2. Batch document embedding.
3. Reranker experiments.
4. SPPE renderer experiments.
5. World-model extractor experiments.
6. Evaluation notebooks.
7. Artifact generation.

Kaggle is not used for live prompt handling, production inference, or urgent chat responses.

13. External Cron

Recommended keep-warm schedule:

Every 5 minutes:

```
GET Lambda /health
GET Render /health
GET Hugging Face /health
```

Every 30 minutes:

```
POST Render /v1/autonomous/ingest-batch
```

POST Render /v1/autonomous/reembed-hash

Every 6 hours:

POST Render /v1/autonomous/evaluate

Daily:

POST Render /v1/autonomous/plan

POST Render /v1/autonomous/report

Weekly or threshold-based:

POST Render /v1/autonomous/kaggle/package

14. Failure Behavior

If the embedding service is down, Lambda should:

1. Use exact retrieval.
2. Use structured retrieval.
3. Use graph retrieval.
4. Use recoverable hash fallback only when necessary.
5. Log fallback state for later re-embedding.
6. Return an answer instead of failing the chat.

If Vectorize is down, Lambda should use Supabase and Neo4j context and lower retrieval confidence.

If Render sleeps, live chat still works because Lambda is the production runtime.

If Kaggle is unavailable, runtime still works and new artifacts wait.

15. Security Rules

Secrets must stay server-side. Never expose these in Vercel public variables:

Supabase service key
Cloudflare API token
Neo4j password
Redis URL
Groq API key
Render agent token
Kaggle token
Hugging Face access token

Real .env files must not be tracked by Git. Only example env files should be committed.

Any secret pasted into chat, screenshots, logs, or command output should be rotated.

16. Production Verification

Live verification performed on June 1, 2026:

```
frontend root: passed
Lambda health: passed
Hugging Face health: passed
sentence-transformer embed: passed, fallback=false, dimension=768
Supabase sign-in through Lambda: passed
live query: passed
chat thread persistence: passed
message order: user then assistant
Lambda training ingest: passed
Render autonomous report: passed
Render autonomous ingest-batch: passed
Render reembed-hash: passed
coding sandbox valid execution: passed
coding sandbox block policy: passed
math solve validation: passed
memory stats endpoint: passed
audit events endpoint: passed
world-model panel: passed
training dashboard: passed
feedback endpoint: passed
learn feedback: passed
unlearn/delete memory: passed
thread delete: passed
```

Earlier during verification, Render protected endpoints returned:

```
503: agent token not configured
```

That was fixed by setting JIMS_RENDER_AGENT_TOKEN in Render and redeploying. Protected endpoints now accept the bearer token and execute bounded work.

17. Implementation Roadmap

Phase 1 complete:

```
Frontend deployed to Vercel
Lambda deployed to AWS
Supabase schema applied
Chat works
Multi-thread persistence works
Feedback/Learn/Unlearn works through Lambda
Hugging Face embedding service returns live sentence-transformer vectors
Render autonomous training endpoints execute with bearer-token protection
Sandbox and math validation paths are exposed and working in production
```

Phase 2 immediate:

```
Add cron jobs
Monitor sentence-transformer reachability
Run reembed-hash on schedule
```

Push health readiness booleans to Render so /health reports secret/config presence

Phase 3:

Strengthen Vectorize recall and retrieval event logging
Expose retrieval misses in the Training UI
Add artifact approval UI
Add rollback controls for active embedding artifacts

Phase 4:

Package Kaggle encoder batches
Train candidate encoder artifacts
Evaluate recall@k, MRR, nDCG, latency, hallucination/error rate
Human-approve artifact activation
Reload embedding service with active approved artifact

Phase 5:

Add reranker artifacts
Move from top-20 retrieval to top-50 plus rerank top-8
Improve dashboards for evaluation and provider degradation

18. Final Direction

JIMS-AI should use frontier models as bounded assistants, not as the owner of memory or truth. JIMS-AI owns:

1. Persistent project memory.
2. Threaded user context.
3. Semantic and exact retrieval.
4. Graph-backed reasoning context.
5. Provenance and trust scoring.
6. Training data collection.
7. Human review and approval.
8. Artifact lifecycle control.

This is the complete v10 direction represented by the current codebase and production deployment.